

High-Performance Computing

Prof. Dr. Ivan Kisel

Robin Lakos, Akhil Mithran, Oddharak Tyagi

High-Performance Computer Architectures

Practical Course

SoSe 2025

Issued: 18.06.2025 / Due: 25.06.2025



Institute for Computer Science

Exercise Sheet 8

Open Computing Language (OpenCL)

This exercise sheet will focus on Open Computing Language (OpenCL), which is a framework for writing programs that execute across heterogeneous platforms consisting of CPUs, GPUs, and other processing units. OpenCL enables efficient parallel computing by utilizing the unique capabilities of different hardware, making it an essential tool for tasks requiring high-performance computation across diverse devices. Its platform-agnostic approach allows developers to write code once and deploy it on various architectures, optimizing resource usage, and boosting processing speeds.

Although OpenCL offers powerful cross-platform computing capabilities, setting it up on a virtual machine (VM) can be complex due to the intricacies of hardware access and the nuances of host and VM configurations. Additionally, installing OpenCL directly on a system also involves careful consideration of hardware compatibility and driver configurations.

In most - if not all - of the machines on site, OpenCL should be installed and configured to run the exercises out of the box. If not, please just switch to another machine on site.

Submission: Submit your source code along with the PDF that includes your analysis.

Exercise 8.1: Matrix Multiplication with OpenCL

We have already seen that matrix multiplication can be computationally expensive, particularly for large matrices. Not only in the area of high-performance computing, GPUs are utilized to gain even more performance by massive parallel computation capacities. The term General Purpose Computation on Graphics Processing Unit (GPGPU) describes the usage of GPUs outside of their actual purpose - graphics processing. For instance, the huge amount of processing elements on a modern GPU can be used to calculate intensive simulations or matrix computations.

If you log in a computer on-site, please check the installation of OpenCL first. You can do this by running the command `clinfo`. In the ideal case, you should get a list of hardware devices, including a GPU (e.g. from Nvidia). If you cannot run it, please ask your tutor for help to find another machine on site that works.

The C++ source code `MatMul.cpp` and the OpenCL kernel code `matrix_multiplication.cl` already compares the OpenCL matrix multiplication with the scalar matrix multiplication. When compiling and running the code, you should see a large difference in runtime. To compile your code use the comment in the source code file.

Run the code and explain the differences of runtime in your PDF.

i) Matrix Addition

Create new files, `MatAdd.cpp` and `matrix_addition.cl`, similar to the matrix multiplication. Implement

the same functionality for matrix addition, including a comparison to a sequential approach, timers, and a result check.

Discuss your results in the PDF. In particular, compare the runtime of your matrix addition to matrix multiplication. For each problem, estimate the total number of floating-point operations required. Also, estimate the total number of elements that must be transferred between RAM and VRAM (host-device memory transfers). Relate the number of floating-point operations to the memory transfer volume for each case (that is, estimate the arithmetic intensity). Based on your findings, explain why GPU acceleration gives a much greater speedup for matrix-matrix multiplication compared to matrix-vector multiplication.

ii) OpenCL Competition

Try to compete with OpenCL. Create and add a new function `FastMatrixMultiplication` to the source code in `MatMul.cpp`, and implement a matrix multiplication that runs as fast as you can tweak it. More precisely, make use of SIMD, OpenMP, TBB or anything else you want to implement that makes sense. Use at least one type each of explicit vectorization and multi-threading and do not forget about compiler flags! It might be helpful to create new data structures with better memory allocation and alignment. You can copy the values of matrices A and B before you measure your time.

Again, measure the time and compare the correctness of results, similar to what is already done for previous exercises.

Additionally, this time, we also want to create some plots for the scalar approach, OpenCL approach, and your approach, each in different colors. Use the matrix size N for the x-axis and the runtime for each approach on the y-axis. Think about the maximum number of $N = 2^i$ that fits into your GPU's VRAM and make a reasonable number of steps starting from $N = 16$. You may use any tool you like to create the plots.

Create a good documentation of your steps, write a small analysis at each meaningful improvement step (improved runtime) in your PDF, e.g. after adding SIMD, after adding OpenMP, and so on.

Finally, analyze the plots in your PDF and explain them. Were you able to beat the GPU computations? Explain when and why.